Categorization Engine — Detailed Documentation Last updated: 2026-07-13 Generated by: AI assistant using Copilot CLI runtime in VS Code

This file provides a deep, technical, and reproducible description of the entire project: deterministic parsing, matcher design, normalization rules, synthetic data generation, ML training pipeline, evaluation metrics and procedures, Streamlit integration, artifacts, and commands to reproduce all steps locally. It is written for engineers who will maintain, extend, or deploy the system.

Table of contents

The system is a hybrid deterministic + ML transaction categorization engine. Deterministic layers (normalizer, parser, matcher, rules, evidence engine) extract structured signals and are prioritized for explainability. A TF-IDF + LogisticRegression model augments recall and coverage where deterministic rules are insufficient. Synthetic data generation was used to rebalance long-tailed categories for model training.

Goals of the design:

- High precision via deterministic rules
- Improved recall via ML augmentation where rules are weak
- Explainability via decision paths, parser rules, and confidence fields
- Reproducible training & evaluation pipelines

2. Full processing flow (step-by-step)

Inputs: an uploaded CSV/Excel with transaction rows (minimum: Narration column)

Step A — Load and normalize

- loader.load_transactions reads Excel/XLSX and returns pandas.DataFrame
- pipeline.process_transactions first normalizes narration using normalize_text (engine/normalizer.py)

Step B — Protocol detection & parsing

- parse_transaction dispatches to one of the rail-specific parsers (UPI, IMPS, NEFT, RTGS, ACH, ATM, CHEQUE or GENERIC)
- Parsers match deterministic regex/token patterns to extract fields: transaction_prefix, transaction_subtype, reference_id, entity_name, bank_name, upi_id, upi_handle, instrument_no
- Parsers set parser_rule and parser_confidence (heuristic values)

Step C — Semantic facts & entity intelligence

- build_semantic_facts consumes parser outputs and applies merchant alias lookups, signal extraction, and intent/movement tagging
- Semantic facts structure contains protocol, entity, movement, intent, bank_family, parser metadata

Step D — Rule-based classification (evidence engine)

- classify_facts (engine/evidence_engine.py) applies category rules and composes a ranked list of candidate categories together with a matched_rule and decision_path
- Outputs final deterministic Category, Confidence, ranked_candidates, alternative_categories, review_required

Step E — (Optional) ML model augmentation

- If enabled (via Streamlit or programmatic call), a pre-trained ML pipeline (TF-IDF + LogisticRegression) predicts categories for normalized narrations
- Hybrid decision strategies may combine rule and ML outputs using parser_confidence thresholds or other policies

Step F — Review & human-in-the-loop

- Streamlit UI exposes parsed fields, classification output, review queue, and (optionally) AI refinement suggestions
- Reviewers correct labels which are stored via training_data.* helpers for future retraining

3. File-by-file responsibilities (detailed)

---

engine/normalizer.py

- normalize_text(text): returns uppercase string, trims whitespace, collapses multi-space, replaces runs of - and / with single '/', returns empty string for NaN
- Regex used: re.sub(r"\s+", " ", text) and re.sub(r"[-/]+", "/", text)

engine/matcher.py

- TOKEN_CHARS constant: 'A-Z0-9_.'
- token_pattern(term): builds boundary-aware regex using token separators set to [\s/_-.:

  (),@]+ and wrap with (?<![{TOKEN_CHARS}]) and (?![{TOKEN_CHARS}]) to prevent partial-token matches
- regex_match(pattern, text): case-insensitive search on normalized text and returns a match metadata dict (rule_name, pattern, source, match, span, groups)
- token_match wraps regex_match on token_pattern
- compact_tokens uses re.findall(r"[A-Z0-9_.]+", normalized)
- clean_entity_text: removes patterns like '(VALUE DATE: ...)', 'VALUE DATE', 'NA', 'UPI' and collapses spaces, trims separators

engine/parser.py

- parse_transaction(row_or_string): routes cash via re.search calls to rail parsers
- parse_upi_transaction: UPI-specific parsing including UPI/REV, REV-UPI, UPI/RRN, and slash-separated UPI family
  - UPI ID extraction regex used: r"[A-Z0-9._+-]{2,}@[A-Z0-9.-]{2,}" (case-insensitive)
- parse_imps_transaction: RECD:IMPS pattern, SENT IMPS compact patterns, IMPSP2A
- parse_neft_transaction: matches r"\bNEFT\s+(.+?)\s+([A-Z]{2,}[A-Z0-9]*[0-9]{4,})\b"
- parse_cheque_transaction: uses negative/positive lookbehind to extract instrument numbers: r"(?<![0-9])([0-9]{2,})(?=[😕\s-])"
- Each parser sets parser_rule and parser_confidence (heuristic: HIGH ~0.88-0.94, MEDIUM ~0.66-0.88, LOW ~0.35-0.40)

engine/rules.py

- load_*_rules: reads JSON files from rules/; expected structure per rule: { "rule_name": "...", "patterns": ["pattern1", "pattern2"], "merchant": "...", // for merchant_rules "mode": "..." // for mode_rules }
- Rules are matched using token_match which in turn uses token_pattern boundary safety

engine/classifier.py

- detect_mode(narration): tests mode_rules, then inferred_modes list, then regex fallback for compact IMPS forms
- detect_merchant(narration): iterates merchant_rules and returns canonical merchant
- classify_transaction(row): builds or uses the semantic facts and delegates to evidence_engine.classify_facts

engine/pipeline.py

- process_transactions(df) performs the orchestration, populates many output columns, runs classify_transaction for each row, and returns an enriched DataFrame with final Category column

app.py (Streamlit)

- Upload Excel -> load_transactions -> process_transactions
- Tabs:
  - Classification Output: shows parser outputs, signals, and classification results
  - Training Review: review queue and saving reviewed labels
  - AI Refinement: hooks for LLM refinements
  - ML Evaluation: load pre-trained pipeline (models/tfidf_logreg_pipeline.pkl), run ML predictions, show metrics, per-class metrics, confusion matrix, mismatches CSV; decision_mode supports ML-only, Rule-only, or Hybrid with parser_confidence threshold

train_and_evaluate.py

- Generates balanced synthetic dataset, trains TF-IDF + LogisticRegression pipeline, saves artifacts to models/

evaluate_sample.py / evaluate_pipeline.py

- Quick evaluation scripts used earlier to run the pipeline on sample/full datasets and compute metrics

4. Normalization rules (exact behavior)

Function: normalize_text(text) — engine/normalizer.py Algorithm (exact):

1. If pandas.isna(text): return ""
2. Convert to string: text = str(text)
3. Uppercase: text = text.upper()
4. Strip leading/trailing spaces: text = text.strip()
5. Collapse multiple whitespace sequences: text = re.sub(r"\s+", " ", text)
6. Collapse multiple consecutive separators '-' or '/' into a single '/': text = re.sub(r"[-/]+", "/", text)
7. return text

Rationale:

- Keep separators normalized so parser can reliably split on '/'.
- Preserve other punctuation like '.' and '_' which may be meaningful in UPI handles, merchant codes, etc.

5. Token matching and matcher internals

Purpose: avoid false positives from substring matches (e.g. 'REV' inside 'FOREVER') and enable robust token/phrase matching across separators.

Key implementation details:

- token_pattern splits the term on separators: re.split(r"[\s/_-.:

  (),@]+", term)
- It builds escaped token pieces and joins them with a permissive separator pattern: r"[\s/_-.:

  (),@]*" (allowing zero-or-more separators between tokens)

- Boundary guards: prefix with (?<![{TOKEN_CHARS}]) and suffix with (?![{TOKEN_CHARS}]) to ensure token boundaries do not occur inside an alphanumeric/._ token
- TOKEN_CHARS was updated to include underscore and dot: 'A-Z0-9_.' to allow handles like 'AMAZON.IN' or 'ICICI_123' to be considered single tokens for boundary detection

Example token pattern generation:

- token_pattern("REV-UPI") -> pieces ['REV', 'UPI'] -> phrase 'REV[\s/-.:

  *(),@]*UPI' -> combined with boundary guards -> (?<![A-Z0-9.])REV[...]*UPI(?![A-Z0-9_.])*

6. Parser internals and key regexes

This section documents the exact regexes used and their behavior.

UPI extraction (in parse_upi_transaction):

- Regex used to extract UPI ID: r"[A-Z0-9._+-]{2,}@[A-Z0-9.-]{2,}" with re.IGNORECASE
  - Explanation: allow letters, digits, dot, underscore, plus and dash in local part; host part allows letters/digits/dot/dash; minimal length 2 for each side
  - This is stricter than generic [\w.-]+@[\w.-]+ and rejects some pathological junk

UPI REV with RRN:

- r"\bUPI/REV\s+([0-9]+)" and r"\bREV[- /]?UPI\b" for multiple forms

UPI RRN with trailing entity:

- r"\bUPI/RRN\s*([0-9]+)\s*/\s*(.*)$" (captures numeric RRN and entity text following slash)

IMPS patterns:

- RECD IMPS pattern: r"\bRECD:?IMPS/([0-9]+)/([^/]+)/([^/]+)(?😕([^/]+))?" captures reference id, entity, bank
- SENT IMPS compact forms: r"\bSENT\s*IMPS*\s([0-9]+)([A-Z0-9 .-]+?)/([A-Z]{3,}[A-Z0-9]*)" *and* r"\bSENTIMPS([0-9]+)([A-Z0-9 .-]+?)/([A-Z]{3,}[A-Z0-9]*)"
- IMPSP2A: r"\bIMPSP2A([0-9]+)\s+(.+)$"

NEFT pattern:

- r"\bNEFT\s+(.+?)\s+([A-Z]{2,}[A-Z0-9]*[0-9]{4,})\b" captures entity and an alphanumeric reference ending with >=4 digits

Cheque extraction:

- r"(?<![0-9])([0-9]{2,})(?=[😕\s-])" captures a numeric instrument_no not preceded by digits and followed by separator

Generic fallbacks:

- Parsers attempt slash-splitting and heuristics (e.g., fullmatch r"[0-9]{6,}" for reference ids) and set parser_confidence accordingly

7. Semantic facts and evidence engine

The semantic facts construction and the evidence engine are split across engine/semantic.py and engine/evidence_engine.py (not fully printed here). The high-level responsibilities:

- build_semantic_facts(row): returns a dict containing protocol, entity (canonical, matched_alias, category), movement, intent, parser metadata
- classify_facts(facts): consumes semantic facts and rule definitions to produce a ranked list of candidate categories, final category, confidence, and decision_path
- Evidence engine design: lightweight signals are used (binary flags like 'bounce', 'charge', 'salary'); these are combined deterministically using rule precedence to compute a category
- Output structure of classify_facts (typical): { 'category': ..., 'confidence': 0.0-1.0, 'decision_path': [...], 'matched_rule': '...', 'ranked_candidates': [{'category':'X','score':0.9}, ...], 'alternative_categories': [...], 'review_required': bool }

8. Rule files and matching behavior

Files: rules/category_rules.json, rules/merchant_rules.json, rules/mode_rules.json

Rule matching principles:

- Rules are lists of patterns. Each pattern is passed to token_match which builds a safe token regex via token_pattern.
- token_match enforces boundary checks so a pattern like 'REV' does not match inside 'FOREVER'.
- Rule precedence: rules are tested in file order; first-match semantics are used in some detection paths (e.g., detect_mode, detect_merchant) to return an immediate canonicalization

Schema conventions (recommended when editing rules JSON):

- rule_name: unique short identifier
- patterns: array of strings; terms that may contain separators or multi-word phrases
- For merchant_rules entries include 'merchant' canonical value; optional 'category' may be included for stronger mapping

9. Synthetic data generation

Script: train_and_evaluate.py

Purpose: rebalance long-tailed label distribution to produce a training set with an approximately equal number of samples per class.

Algorithm (detailed pseudocode):

1. Read original labeled rows: normalized narration (strings) and labels
2. Count per-class occurrences; let max_count = max(counts)
3. Set target_per_class = min(max_count, SYNTHETIC_PER_CLASS_CAP)
4. For each label L with samples S: if len(S) >= target_per_class: randomly sample target_per_class examples from S else: generated = list(S) loop until len(generated) >= target_per_class: src =

random.choice(S) aug = augment_text(src) # see augmentation below if aug not in generated: generated.append(aug) safety: break after many iterations and allow duplicates to avoid infinite loops

5. Combined generated samples across all classes -> balanced dataset

augment_text(text) (exact implementation):

- normalize_text(text) to ensure consistent case/separators
- tokens = re.split(r"([\s/_-:.@,]+)", normalized) # keeps separators tokens
- word_indices = positions of word tokens (even indices)
- With chance 0.12, delete a randomly selected word token
- With chance 0.08, swap two adjacent word tokens
- For each separator (odd index), with chance 0.15, replace it with a random choice among ['/', '-', ' ', ' / ']
- With chance 0.06, append ' / ' + random integer between 100 and 99999
- Re-join tokens and collapse multiple spaces

Parameters used in session run:

- SYNTHETIC_PER_CLASS_CAP = 2000
- RANDOM_SEED = 42
- Augmentation probabilities: deletion 0.12, swap 0.08, separator change 0.15, numeric noise 0.06

Limitations and caveats:

- Augmentations are lightweight and token-level; they do not synthesize fully realistic merchant names or protocol-specific token structures, so synthetic samples may be less realistic than true production variants.
- Synthetic balancing can cause the model to overfit to augmented artifacts. Always evaluate on a held-out real dataset.

10. ML training pipeline (exact code & hyperparameters)

Script: train_and_evaluate.py (core snippet)

Preprocessing and splitting (exact):

- X = out_df['Narration'].astype(str).to_numpy()
- y = out_df['Label'].astype(str).to_numpy()
- X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, stratify=y, random_state=RANDOM_SEED)

Model pipeline (exact): from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.linear_model import LogisticRegression from sklearn.pipeline import Pipeline vec = TfidfVectorizer(ngram_range=(1,2), min_df=2) clf = LogisticRegression(max_iter=2000) pipeline = Pipeline([('tfidf', vec), ('clf', clf)])

Training:

- pipeline.fit(X_train, y_train)

Evaluation:

- y_pred = pipeline.predict(X_val)

- Accuracy/precision/recall/f1 weighted computed via sklearn.metrics

Artifacts saved:

- models/tfidf_logreg_pipeline.pkl (pickle.dump(pipeline))
- models/train_balanced.csv (balanced dataset CSV)
- models/confusion_matrix.json (JSON form of confusion matrix saved during training)

Hyperparameters used:

- TF-IDF: ngram_range=(1,2), min_df=2
- LogisticRegression: default penalty 'l2', solver = liblinear or default (scikit-learn picks), max_iter=2000
- Random seed: 42

Recommendations for tuning:

- Try logistic regression penalty solver adjustments or C regularization: grid-search on C in [0.01, 0.1, 1, 10]
- Consider CalibratedClassifierCV for probability calibration
- Consider TF-IDF min_df tuning or char ngrams for noisy IDs

11. Evaluation metrics (formal definitions)

These are the metrics implemented and exposed by the app and scripts.

Let TP_c, FP_c, FN_c be true positives, false positives, false negatives for class c.

Precision_c = TP_c / (TP_c + FP_c) if denominator > 0 else 0 Recall_c = TP_c / (TP_c + FN_c) if denominator > 0 else 0 F1_c = 2 * Precision_c * Recall_c / (Precision_c + Recall_c) if sum > 0 else 0

Support_c = number of true instances for class c

Weighted averages (used by default): Precision_weighted = sum(Precision_c * Support_c) / sum(Support_c) Recall_weighted = sum(Recall_c * Support_c) / sum(Support_c) F1_weighted = sum(F1_c * Support_c) / sum(Support_c)

Accuracy = (sum over all classes of TP_c) / total_examples

Confusion matrix: matrix M where M[i, j] is the count of samples with true label labels[i] predicted as labels[j]

When to prefer which metric:

- For imbalanced datasets: weighted precision/recall/F1 give an overall sense but per-class recall is critical when missing certain classes is costly (e.g., reversals, fraud signals)
- If costs are asymmetric (false negative more costly), focus on per-class recall and choose thresholds accordingly

12. Evaluation procedures, calibration, validation strategy

Recommended reproducible evaluation pipeline:

1. Hold out a purely real test set (no synthetic augmentation) — this is the canonical evaluation for production readiness
2. During training: use stratified train/val split (80/20) from the balanced training set or use cross-validation with folds stratified by label
3. For numeric stability and probability interpretability, calibrate probabilities if you rely on predict_proba for thresholds: from sklearn.calibration import CalibratedClassifierCV base_clf = LogisticRegression(max_iter=2000) calibrated = CalibratedClassifierCV(base_clf, cv='prefit' or 5) calibrated.fit(X_train_transformed, y_train)
4. Select final decision thresholds using validation set to trade off precision/recall per class, or use cost-sensitive training
5. For deployment: prefer hybrid model where rule outputs are trusted when parser_confidence >= high_threshold and ML used otherwise. This preserves explainability and deterministic guarantees.
6. Streamlit app integration details

---

app.py features added by this session:

- ML Evaluation tab with decision_mode choices and detailed error report generation
- Ability to download per-class metrics CSV, confusion matrix CSV, and mismatches CSV

State management recommendations:

- Use st.session_state to store processed_df (original), and a processed_df_with_ml when ML 'Apply' is used
- Downstream tabs (Training Review, AI Refinement) should refer to session_state.get('processed_df_with_ml', processed_df) to avoid re-running the model unnecessarily

Apply ML to session behavior (recommended safe pattern):

- Create Final Category column and keep original Category unchanged unless an explicit overwrite checkbox is selected
- Persist only to session_state; explicit 'Download' or 'Save to disk' action required to write overwritten labels out

14. Artifacts produced and file formats

- models/train_balanced.csv : CSV with columns 'Narration', 'Label' (UTF-8)
- models/tfidf_logreg_pipeline.pkl : Pickle of sklearn Pipeline (TF-IDF + LogisticRegression). Use Python/pickle to load.
- models/confusion_matrix.json : JSON with 'labels': [...], 'matrix': [[...]]
- DOCUMENTATION.md, DOCUMENTATION_DETAILED.md : project docs

15. Reproducible commands and scripts

Prerequisites: Python 3.8+, pip Install deps: python -m pip install --upgrade pip python -m pip install pandas scikit-learn streamlit numpy

Run training (synthetic + train): python train_and_evaluate.py

Run sample evaluation (fast): python evaluate_sample.py

Run full evaluation (may take long): python evaluate_pipeline.py

Start Streamlit UI: streamlit run app.py

If Streamlit cannot import sklearn, ensure you installed it in the interpreter used to start streamlit. In the same terminal, run: python -c "import sys; print(sys.executable); print(sys.version)" python -m pip show scikit-learn where streamlit

16. Troubleshooting and common pitfalls

Q: Streamlit reports "No module named 'sklearn'" even after installation A: Streamlit uses a different Python interpreter/environment. Run python -c "import sys; print(sys.executable)" in the terminal and pip install scikit-learn into that interpreter using python -m pip install scikit-learn

Q: Model shows near-perfect validation accuracy after synthetic balancing A: This is expected: synthetic augmentation inflates apparent validation performance. Always evaluate on a held-out real dataset.

Q: train_test_split fails due to pyarrow-backed pandas Series A: Convert pandas Series to numpy arrays before calling scikit-learn functions (astype(str).to_numpy()). This was fixed in train_and_evaluate.py

Q: Deterministic rules misclassify due to substring matches A: Edit rules/ JSON patterns and rely on token_match (token_pattern boundary behavior) to craft safe rule patterns. Increase coverage by adding multi-token patterns and canonical aliases.

Q: UPI extraction misses handles or returns junk A: The UPI regex is intentionally stricter. If you need more flexibility, relax the local-part rules but maintain anchors and validate using known handle patterns.

17. Recommended next work items and improvements

- Add CalibratedClassifierCV for probability calibration and expose probability thresholds in the UI
- Implement Apply-ML-to-session feature with safe overwrite and session_state wiring (I can implement this)
- Add automated rule-suggestion pipeline: mine mismatches and propose candidate rule patterns (semi-automated)
- Expand augmentation to merchant-aware substitutions using merchant alias lists to create more realistic synthetic examples
- Add continuous evaluation and drift detection via scheduled runs on production streams

18. Change log & provenance

- 2026-07-13: Created DOCUMENTATION and DOCUMENTATION_DETAILED.md by AI assistant using Copilot CLI runtime

- Modifications made during the session (files edited/created):
  - engine/matcher.py (TOKEN_CHARS, compact_tokens change)
  - engine/parser.py (tightened UPI regex)
  - engine/normalizer.py (separator handling)
  - evaluate_pipeline.py, evaluate_sample.py added
  - train_and_evaluate.py added (synthetic generation and training)
  - app.py updated with ML Evaluation tab & detailed report options
  - models/ artifacts created during training: train_balanced.csv, tfidf_logreg_pipeline.pkl, confusion_matrix.json

Provenance: this document was created programmatically by the assistant and reflects the current codebase state and the training/evaluation runs executed in this session. Always validate file paths and environment specifics on your machine before deploying.

Appendix: Useful code excerpts

1. TF-IDF + LogisticRegression pipeline (exact code used in training)

from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.linear_model import LogisticRegression from sklearn.pipeline import Pipeline vec = TfidfVectorizer(ngram_range=(1,2), min_df=2) clf = LogisticRegression(max_iter=2000) pipeline = Pipeline([('tfidf', vec), ('clf', clf)])

2. UPI ID regex used for extraction: [A-Z0-9._+-]{2,}@[A-Z0-9.-]{2,}

3. Token pattern generation (pseudocode): pieces = [re.escape(piece) for piece in re.split(r"[\s/-.:

   (),@]+", term) if piece] phrase = r"[\s/-.:

   (),@]*".join(pieces) pattern = rf"(?<![{TOKEN_CHARS}]){phrase}(?![{TOKEN_CHARS}])"

End of document.